

Software Engineering

Introduction

Okba Tibermachine
National School of Artificial Intelligence

September 30, 2025

Contents

1. Programming VS Software Engineering
2. Why Software Engineering ?
3. Definition and Historical Context
4. No Silver Bullet
5. Topics of Study in Software Engineering
6. Software Processes
7. Goals and Qualities of good software product
8. Principals of Software Engineering
9. Classification of Software Systems
10. Team Role in SE Team

Programming VS Software Engineering

A Simple Challenge

You are asked by a client to write a **"Program"** to print **"Hello World"** to the console using the Java Programming Language.

Task: Write the code that accomplishes this.

Take 30 seconds to think about your solution...

The "Obvious" Solution

What most programmers would write:

```
1 public class Hello {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World");  
4     }  
5 }
```

Compilation and execution:

- Java Development Kit (JDK) must be installed
- To compile: `javac Hello.java`
- To execute: `java Hello`

Question: Does this solve the problem?

The "Obvious" Solution

What most programmers would write:

```
1 public class Hello {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World");  
4     }  
5 }
```

Compilation and execution:

- Java Development Kit (JDK) must be installed
- To compile: `javac Hello.java`
- To execute: `java Hello`

Question: Does this solve the problem?

Yes, technically
But no, it does not solve the real problem
The real problem is to
deliver a piece of software that has lasting value,
can adapt to change,
and can be understood by others.

What's Wrong With That Obvious Code?

Is this good programming practice?

- Putting Business Logic inside the main procedure
- No use of Object-Oriented Design
- Inclusion of Text inside the code
- No comments
- No Abstraction
- Not Modular

```
1 public class Hello {  
2     public static void main(String[] args)  
3         System.out.println("Hello World")  
4     }  
5 }
```

A More Engineered Solution

```
1  /**
2   * This class would print a hello world message by creating a class with a single print method
3   * @author Imed Bouchrika
4   * @version 2023.02.14
5   */
6  public class Hello{
7      /**
8       * The method print prints to the console the message Hello World
9       * @return returns True when the message is printed successfully.
10      */
11     public boolean print(){
12         System.out.println("Hello World");
13         return true;
14     }
15     /**
16      * This is the main method to be invoked when launching the program
17      * @param args are the command line arguments
18      */
19     public static void main(String [] args){
20         Hello myhello =new Hello();
21         myhello.print();
22     }
23 }
24
```


The Real Questions Begin

Now let's think beyond just "writing code"...

1. What if the customer wants to change "Hello World" to "Hello All"?
(**Requirements Change**)
2. How much do you charge the customer for this? (**Business Considerations**)
3. Will you be ethically required to assist, train, and support the customer?
(**Professional Responsibility**)
4. What if they want it in multiple languages? Or different output formats?
or(**Scalability**)
5. Can you still maintain and work on this code after 10 years? (**Maintenance**)

The Real Questions Begin

Now let's think beyond just "writing code"...

1. What if the customer wants to change "Hello World" to "Hello All"?
(**Requirements Change**)
2. How much do you charge the customer for this? (**Business Considerations**)
3. Will you be ethically required to assist, train, and support the customer?
(**Professional Responsibility**)
4. What if they want it in multiple languages? Or different output formats?
or(**Scalability**)
5. Can you still maintain and work on this code after 10 years? (**Maintenance**)

These questions reveal the gap between programming and software engineering.

Computer Program & Software

This simple example reveals three key distinctions:

Computer Program

A set of instructions written in a programming language that will be compiled or translated to machine language to perform a specific task.

Software

The set of computer programs **in addition to** the associated documentation, configuration files, and data.

Software product

The product is an **Industrial-strength** software of *high quality* to *solve some problem* of some *set of users*

Ad-hoc Programming Vs Software Engineering

Ad-hoc Programming:

- Write computer programs in an unplanned way to solve particular problems
- Main focus is on starting the programming task immediately
- "Just make it work"

Ad-hoc Programming Vs Software Engineering

Ad-hoc Programming:

- Write computer programs in an unplanned way to solve particular problems
- Main focus is on starting the programming task immediately
- "Just make it work"

Software Engineering:

- Systematic, disciplined, and quantifiable approach
- Consider long-term maintainability, scalability, and quality
- Plan for change, documentation, testing, and professional practices
- "Make it work, make it right, make it maintainable" = Make it Valuable.

Analogy



Why Software Engineering ?

Why Software Engineering ?

"Our civilization runs on software."

— Bjarne Stroustrup

Software is everywhere:

- Organizations depend on software systems to deliver services
- Governments interact with citizens through software applications
- E-commerce platforms enable global business
- Software is embedded in cars, airplanes, satellites, medical devices
- Traditional industries are being transformed by software

Why Software Engineering ?

Software Engineering Failure is expensive !

Global Impact

- 68% of SE projects fail in some way (Choas Report 2009)

Types of failure:

- Cancelled projects
- Delivered late
- Run over budget
- Don't meet requirements

SE Failure examples: Ariane 5 Rocket (1996)

When a few lines of code cost \$500 million:

- **The incident:** French rocket exploded 30 seconds after launch
- **The cause:** Software defect in onboard system
- **The code:** Converting 64-bit floating-point to 16-bit integer
- **The problem:** Conversion worked in testing and previous launches
- **The failure:** Launch day produced a number too large for 16-bit integer

The Lesson

Even a few lines of code, when not properly engineered for all possible conditions, can have catastrophic consequences.

SE failure examples: UK NHS NPfIT (2002-2011)

The largest IT project failure in history:

- **Project:** National Programme for IT in the NHS
- **Goal:** Modernize UK's National Health Service IT systems
- **Duration:** 9 years
- **Cost:** £12 billion
- **Outcome:** Cancelled with minimal deliverables

Official Statement

"The government will formally announce the scrapping of the National programme for IT in the NHS... There can be no confidence that the programme has delivered or can be delivered as originally conceived."

Why Software Engineering Matters

Given this vital role, we need systematic approaches to build software systems

Definition and Historical Context

What is Software Engineering?

Definition

Software engineering focuses on **systematic**, **disciplined**, and **quantifiable** approaches to developing, operating, maintaining, and evolving software systems.

Key aspects:

- **Systematic**: using organized methods and processes rather than ad-hoc coding
- **Disciplined**: developers should follow rules, standards, and best practices.
- **Quantifiable**: progress, quality, and performance should be measured and evaluated.

Historical Context

- **Late 1960s:** Software engineering field emerged
- **Before:** Focus was on building machines for scientific problems
- **Hardware advances:** Computers became more common in universities and businesses
- **New challenges:** Users demanded more complex applications
 - Commercial systems (payroll, accounting, inventory)
 - More diverse and sophisticated requirements

The NATO Conference (1968)

- **October 1968:** 50 computer scientists met in Garmisch, Germany
- **Purpose:** Discuss the "software crisis" ^a
- **Outcome:** 130-page report advocating for software development based on engineering principles
- **Significance:** Landmark in establishing software engineering as a discipline

^aSoftware Crisis : Difficulty writing useful and efficient computer programs in the required time.



*Photo of scientists at the 1968
NATO conference*

NATO Conference Challenges

Key Quote from Conference

"The basic problem is that certain classes of systems are placing demands on us which are beyond our capabilities and our theories and methods of design and production at this time... It is large systems that are encountering great difficulties."

Definition : A discipline that integrates processes, methods and tools for the analysis, design, development, testing and maintenance of computer software products whilst complying with the set of conventional principles and best practices.

No Silver Bullet

Frederick Brooks and "No Silver Bullet"

- **Frederick Brooks:** 1999 Turing Award recipient, Software Engineering pioneer
- **1987 Essay:** "No Silver Bullet: Essence and Accident in Software Engineering" => There is no single magical solution ("silver bullet") that will suddenly make software development easy, cheap, fast, and error-free.
- **Main Argument:** Software has unique characteristics compared to other engineering products

Two Types of Difficulties

1. **Essential Difficulties:** Intrinsic to the field, unlikely to be solved by new technologies
2. **Accidental Difficulties:** Can be addressed with adequate training and existing technologies

Essential Difficulties in Software

1. Complexity

- Software is among the most complex human constructions.
- Even traditional engineering products increasingly rely on software

2. Conformity

- Software must adapt continuously to changing environments
- Example: Tax legislation changes require prompt software adaptations
- Unlike physics, where natural laws don't change due to human decisions

Essential Difficulties (continued)

3. Changeability

- Constant pressure for software to evolve and add new features
- More successful software → more modification requests
- Unlike cars: once manufactured, they don't usually receive new functionalities

4. Invisibility

- Inherently challenging to visualize software structure and complexity
- Abstract nature makes evaluation difficult
- Unlike cars: physical properties (weight, height, shape) are visible

Accidental vs Essential Difficulties

Accidental Difficulties (Solvable)

- IDE that frequently crashes
- Compiler with cryptic error messages
- Framework lacking documentation
- Web application with confusing interface

Essential Difficulties (Inherent)

- Software complexity
- Need for conformity to changing environments
- Pressure for changeability
- Invisibility of software structure

Real World Example: Software Scale

The complexity of software systems can be illustrated by their size:

- **Linux Foundation projects:** Over 1.7 billion lines of code
- **Contributors:** 65,000 active developers
- **Linux kernel:** More than 30 million lines of code
- **Google systems:** Billions of lines of code

Source: Linux Foundation 2023 report

More: Millions of Lines of Code

Topics of Study in Software Engineering

SWEBOK: Software Engineering Body of Knowledge

- **Guide to SWEBOK:** Comprehensive report by IEEE Computer Society Link
- **Purpose:** Document the body of knowledge defining software engineering
- **Created by:** Numerous researchers and industry professionals
- **Internationally recognized:** Scientific society standards

Defines 18 key areas of Software Engineering



*Guide to the Software
Engineering Body of Knowledge*

Editors

Pierre Bourque
Richard E. (Dick) Fairley



IEEE  computer society

SWEBOK 2024 Key Areas (Part 1)

1. **Software Requirements** – Eliciting, analyzing, and managing what software must achieve.
2. **Software Architecture** – Defining the system's high-level structure and components.
3. **Software Design** – Refining architecture into detailed solutions and interfaces.
4. **Software Construction** – Coding, unit testing, and integrating components.
5. **Software Testing** – Verifying and validating software against requirements.

SWEBOK 2024 Key Areas (Part 2)

6. **Software Engineering Operations** – Ensuring reliable, efficient operation in production.
7. **Software Maintenance** – Modifying software post-delivery to fix or improve it.
8. **Software Configuration Management** – Controlling versions, changes, and builds.
9. **Software Engineering Management** – Planning, leading, and coordinating software projects.
10. **Software Engineering Process** – Defining and improving development approaches.

SWEBOK 2024 Key Areas (Part 3)

- 11. **Software Engineering Models and Methods** – Applying formal methods and modeling techniques.
- 12. **Software Quality** – Ensuring reliability, performance, and usability standards.
- 13. **Software Security** – Protecting software against threats and vulnerabilities.
- 14. **Software Engineering Professional Practice** – Ethical, legal, and teamwork practices.
- 15. **Software Engineering Economics** – Analyzing costs, benefits, and trade-offs.

SWEBOK 2024 Key Areas (Part 4)

- 16. **Computing Foundations** – Core concepts of computer science essential for software.
- 17. **Mathematical Foundations** – Logic, algebra, and discrete math applied to software.
- 18. **Engineering Foundations** – Principles of engineering applied to software systems.

Requirements Engineering

Definition

Requirements define both **what** a system should do and **how** it should behave.

Requirements Engineering Activities:

- Elicit requirements
- Analyze requirements
- Document requirements
- Validate requirements

Types of Requirements

Functional Requirements

- Define features or services
- What the system should provide

Banking App Examples:

- Account balance display
- Statement generation
- Transfers between accounts
- Debit card cancellation

Non-Functional Requirements

- How the system should operate
- Constraints and quality of service

Examples:

- Performance
- Availability
- Security
- Usability
- Scalability
- Maintainability

Non-Functional Requirements Examples

Banking Application Non-Functional Requirements:

- **Performance:** Provide account balance within 2 seconds
- **Availability:** Online 99% of the time
- **Fault tolerance:** Continue functioning if a data center fails
- **Security:** Encrypt all data exchanged with branches
- **Privacy:** Keep customer data confidential
- **Interoperability:** Integrate with Central Bank systems
- **Scalability:** Handle data for 1 million customers
- **Usability:** Accessible to visually impaired individuals

Software Design

Definition

Software design involves determining the principal code units of a software system, extending only to the level of interfaces.

Key Concepts:

- **Provided Interfaces:** Services that a code unit makes public
- **Required Interfaces:** Services that a code unit depends on
- **Implementation details:** Not addressed during design phase

Design Example: Bank Account

```
1 class BankAccount {  
2     private Customer customer;  
3     private double balance;  
4  
5     public double getBalance() { ... }  
6     public String getCustomerName() { ... }  
7     public String getStatement(Date start, Date end) { ... }  
8     ...  
9 }
```

- **Provided Interface:** Public methods (getBalance, getCustomerName, etc.)
- **Required Interface:** Customer class
- **Dependency:** BankAccount depends on Customer

Architectural Design

Definition

When design becomes more abstract and involves larger units like packages or folders, it is called **architectural design**.

Software Architecture:

- Organization of a system at higher level of abstraction
- Beyond individual classes or code units
- Focuses on system structure and component relationships
- Defines overall system organization

Software Construction

Definition

Construction refers to the implementation phase—programming the system.

Key Decisions During Construction:

- Algorithms and data structures to use
- Installing and configuring third-party frameworks and libraries
- Exception handling policies
- Coding standards (naming conventions, indentation, documentation)
- Development tools (compilers, IDEs, debuggers, version control)
- **Since 2020:** AI-based tools and Large Language Models (LLMs)

Software Testing

Definition

Testing involves executing a program with a finite set of cases and verifying whether it delivers the expected results.

Dijkstra's Quote (1972 Turing Award)

"Testing shows the presence, not the absence of bugs."

- Testing can only prove that something is wrong, not that everything is right.
- Programs are too complex to test exhaustively.
- Absence of evidence is not evidence of absence.

Types of Testing

- **Unit Tests:** Testing small code units (like a class)
- **Integration Tests:** Testing larger units (set of classes)
- **Performance Tests:** Checking system performance under specific loads
- **Usability Tests:** Evaluating user interface usability

Testing serves two purposes:

1. **Verification:** Are we building the product right? (according to specification)
2. **Validation:** Are we building the right product? (meets customer needs)

Software Maintenance

modifying and updating a software system after its delivery to:

- correct faults (fix bugs),
- improve performance or other qualities,
- adapt the software to a changed environment (new OS, hardware, regulations),
- or add new features to meet evolving user needs.

Software Maintenance Categories

Software systems require maintenance like traditional engineering systems:

1. **Corrective:** Address bugs reported by users/developers
2. **Preventive:** Address latent bugs that haven't caused failures
3. **Adaptive:** Adjust system to environmental changes
4. **Refactoring:** Improve design without changing external behavior
5. **Evolutionary:** Add new features or enhance existing ones

Maintenance Examples

Preventive Maintenance: Y2K Problem

- **Issue:** Applications used DD-MM-YY date format
- **Concern:** Date operations in 2000+ could produce incorrect results
- **Solution:** Convert to DD-MM-YYYY format
- **Scale:** Massive global effort by organizations

Adaptive Maintenance Examples:

- Updating from Python 2.7 to Python 3.0
- Customizing system for new customer requirements
- Modifying system to comply with new legislation

Project Management

the process of planning, organizing, directing, and controlling resources (people, time, budget, tools, and technology) to successfully complete a software project within defined scope, time, cost, and quality constraints. **Activities Include:**

- **Planning** : Defining objectives, tasks, schedules, resources, and risks.
- **Organizing** : Assigning roles, responsibilities, and structuring the team.
- **Execution** : Coordinating the team's work to build the software.
- **Monitoring Controlling** : Tracking progress, handling changes, managing risks, and ensuring quality.
- **Closing** : Delivering the final product, evaluating performance, and learning for future projects.

Brooks' Law

Brooks' Law

"Adding manpower to a late software project makes it later."

Rationale:

- New developers need time to understand codebase, architecture, design
- Larger teams require more communication and coordination
- Communication complexity grows exponentially

Communication Channels:

- 3 developers: 3 channels
- 4 developers: 6 channels
- 10 developers: 45 channels

Solution: Develop in small teams (≤ 12 engineers)

Software Processes

Software Process Definition

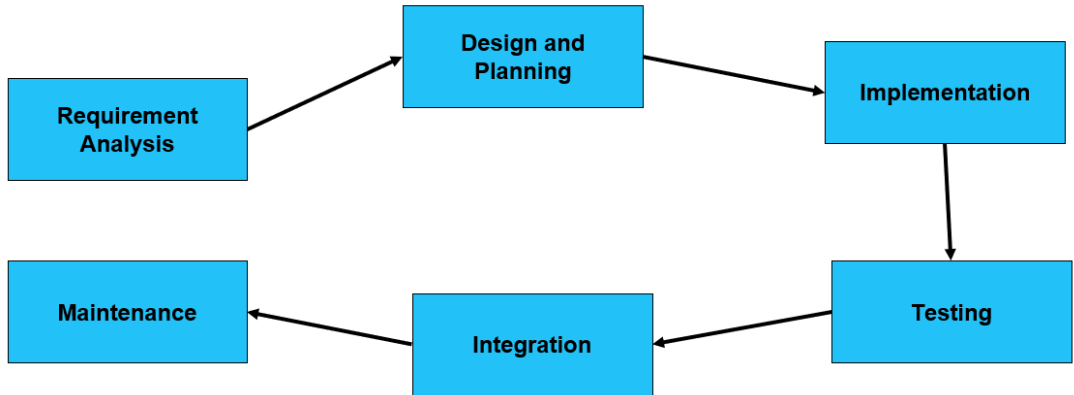
Definition

A software process defines the structured sequence of activities and events required to build, test, and deliver software.

Analogy: Like building construction

- Foundation → Masonry → Roofing → Plumbing → Electrical → Painting
- Requirement Analysis → Design and Planning → Implementation → Testing → Integration → Maintenance

Software Process



Two Main Process Types in Software Engineering

Plan-Driven Processes

- All activities are planned in advance
- Progress measured against the plan
- Emphasis on documentation
- Requirements defined upfront
- Sequential phases
- Example: Waterfall Model

Agile Processes

- Planning is incremental
- Easier to change process
- Less documentation
- Requirements evolve
- Iterative development
- Example: Scrum, XP

Key Difference: Plan-driven processes work best when requirements are stable and well-understood, while Agile processes are better suited for projects with changing requirements.

Goals and Qualities of good software product

Goal

The goal of software development is to develop :

- **High quality** software that meets customers' real needs.
- **On time**
- **On budget**

Qualities of good software product

- What are the qualities of good software product with **industrial-strength**
1. **Correctness (Functionality):** The software meets the requirements and specifications that it was designed for, and it behaves as expected when it is used in its intended environment.
All functionalities are correctly implemented

Qualities of good software

- What are the qualities of good software product with **industrial-strength**
2. **Usability and Acceptability:** The software is easy to use and understand, and it provides a positive user experience.
Users should also accept and adopt the software product by continuing to use it

Qualities of good software

- What are the qualities of good software product with **industrial-strength**
3. **Reliability:** The software is free of defects and it performs consistently and accurately under different conditions and scenarios.

Qualities of good software

- What are the qualities of good software product with **industrial-strength**
4. **Maintainability:** The software is easy to change and update, and it is well-documented, so that it can be understood and modified by other developers.

Qualities of good software

- What are the qualities of good software product with **industrial-strength**
5. **Security:** The software is protected against unauthorized access and it keeps the data and functions safe from malicious attacks.

Qualities of good software

- What are the qualities of good software product with **industrial-strength**
6. **Efficiency and Performance:** The software runs efficiently and quickly, and it can handle large amounts of data or traffic.

Qualities of good software

- What are the qualities of good software product with **industrial-strength**
7. **Scalability:** The software can handle an increasing workload and it can be easily extended to meet the changing requirements.
Scaling to increase large number of users, big data or even larger spatial deployment

Qualities of good software

- What are the qualities of good software product with **industrial-strength**
8. **Portability:** the ease with which software can be used on computer or platform configurations other than its current one where it was developed and tested.

Qualities of good software

- What are the qualities of good software product with **industrial-strength**

9. **Reusability and Openness:** the ease with which software can be reused in developing other software.

Qualities of good software

- What are the qualities of good software product with **industrial-strength**
10. **Adaptability and Flexibility:** The software is flexible to be customized without extra development in order to adapt it to different future requirements.

Principals of Software Engineering

Principles of Software Engineering

- There is no standard set of universal principles and laws in software engineering that is agreed to by everyone.
- Principles are suggested to serve as guidelines and recommendations to improve the productivity of software development and achieve a better success.

Principles of Software Engineering

- Davis's principles (1994):
 1. Make quality Number 1
 2. High Quality Software is possible
 3. Give products to customer early
 4. Determine the Problem before the requirements
 5. Evaluate design alternatives
 6. Use an appropriate process model
 7. Use different languages for different phases
 8. Minimize intellectual distance
 9. Put technique before tools
 10. Get it right before you make it faster
 11. Inspect Code
 12. Good management is more important than technology
 13. People are the key to success
 14. Follow with care
 15. Take responsibility

Intellectual Distance
defined as the gap between
the real world and computerized world

Principles of Software Engineering

- Davis proposed another complementary 15 recommendations for Software Engineering Professionals including:
 - Understand the Customers' Priorities
 - The More They See, the More They Need
 - Plan to Throw One Away.
 - Design for Change
 - Design Without Documentation is Not Design
 - Avoid Tricks
 - Don't Test Your Own Software

Principles of Software Engineering

- There are other principles and recommendations including the following:
 1. Wasserman's 8 Concepts
 2. Royce's 10 principles,
 3. David Hooker's 7 Principles

Classification of Software Systems

Bertrand Meyer's ABC Classification

Three Primary Types of Software Systems:

1. **Type A:** Acute Systems
2. **Type B:** Business Systems
3. **Type C:** Casual Systems

Each type requires different software engineering approaches and practices.

Type C: Casual Systems

Characteristics

- Not under pressure for high-quality performance
- Can tolerate minor bugs without operational jeopardy
- Lightweight and non-critical
- Typically implemented by single developer

Examples:

- Academic project scripts
- One-time data migration programs
- Student association membership management systems

Engineering Approach: Do NOT over-engineer these systems

- Don't require high internal quality standards
- Don't need optimal runtime performance
- Don't need sophisticated user interfaces

Type A: Acute Systems

Critical Characteristic

A single failure can have catastrophic consequences, including loss of human lives.

Examples:

- Autonomous vehicle control systems
- Nuclear power plant control systems
- Airplane control systems
- Intensive Care Unit medical equipment (Ventilators, Patient Monitors, Dialysis machines ...etc)
- Subway train control systems
- Ariane 5 rocket software (discussed earlier)

Type A: Development Requirements

Rigorous Development Process:

- Comprehensive code reviews
- External certification required
- Hardware and software redundancies

Design Approaches:

- Formal languages based on logic or set theory
- Mathematical proofs of correctness
- Extensive testing and validation

Note

Type A (Acute) systems development has other methodologies and techniques beyond this course.

Type B: Business Systems

Examples:

- Enterprise applications (finance, HR, logistics, sales)
- Web-based systems
- Software libraries and frameworks
- General-purpose applications (text editors, spreadsheets)
- Basic software systems (compilers, IDEs)

System Classification Summary

Type	Failure Impact	Examples	Engineering Level
A (Acute)	Life-threatening	Aircraft control	Extremely rigorous
B (Business)	Business impact	Enterprise apps	Standard practices
C (Casual)	Minimal impact	Academic scripts	Minimal engineering

Key Insight: Match engineering effort to system criticality

- Over-engineering Type C systems wastes resources
- Under-engineering Type A systems risks lives
- Type B systems: Sweet spot for standard SE practices

Why Classification Matters

Important Point

There is no single, universal method for software development. Different types of software require different approaches.

Software varies in:

- Size and complexity
- Functional and non-functional requirements
- Development processes needed
- Quality assurance practices required

Team Role in SE Team

Software Development Team Roles

- Project Manager
- Product Manager
- Software Architect
- UX/UI Designer
- Software Developer
- QA Engineer
- DevOps Engineer
- Business Analyst

Project Manager

Key Responsibilities:

- Plans, organizes, and manages the entire software development process
- Defines project scope, timeline, and budget
- Coordinates team members and resources
- Manages risks and resolves issues
- Ensures project delivery on time and within budget
- Acts as liaison between stakeholders and development team

Skills: Leadership, communication, time management, problem-solving

Product Manager

Key Responsibilities:

- Defines the product vision and strategy
- Prioritizes features based on business value
- Gathers and analyzes user requirements
- Creates product roadmap
- Works with stakeholders to align product with business goals
- Makes decisions on what to build and when

Skills: Strategic thinking, market analysis, user empathy, decision-making

Software Architect

Key Responsibilities:

- Designs the overall technical structure of the system
- Makes high-level design choices
- Defines coding standards and tools
- Ensures scalability, security, and performance
- Reviews technical implementations
- Provides technical guidance to developers

Skills: Deep technical expertise, system design, architectural patterns

UX/UI Designer

Key Responsibilities:

- Creates user-friendly interfaces
- Conducts user research and testing
- Designs wireframes, mockups, and prototypes
- Ensures consistent visual design across the product
- Optimizes user experience and interaction flows
- Collaborates with developers on implementation

Skills: Design tools (Figma, Sketch), user research, visual design, empathy

Software Developer / Programmer

Key Responsibilities:

- Writes, tests, and maintains code
- Implements features according to specifications
- Debugs and fixes software issues
- Collaborates with team members on technical solutions
- Participates in code reviews
- Creates technical documentation

Types: Frontend, Backend, Full-Stack, Mobile, etc.

Skills: Programming languages, frameworks, algorithms, problem-solving

QA Engineer / Tester

Key Responsibilities:

- Develops and executes test plans and test cases
- Identifies, documents, and tracks bugs
- Performs manual and automated testing
- Ensures software quality and reliability
- Verifies fixes and validates features
- Conducts regression, performance, and security testing

Skills: Testing methodologies, automation tools, attention to detail

DevOps Engineer

Key Responsibilities:

- Manages deployment pipelines and infrastructure
- Automates build, test, and deployment processes
- Monitors system performance and availability
- Manages servers, databases, and cloud resources
- Implements security best practices
- Facilitates collaboration between development and operations

Skills: CI/CD tools, cloud platforms, scripting, system administration

Business Analyst

Key Responsibilities:

- Bridges the gap between business and technical teams
- Gathers and documents business requirements
- Analyzes business processes and workflows
- Creates functional specifications
- Validates that solutions meet business needs
- Facilitates communication with stakeholders

Skills: Requirements analysis, documentation, communication, domain knowledge

Team Collaboration

Successful software development requires:

- Clear communication between all roles
- Shared understanding of goals and priorities
- Respect for each role's expertise
- Collaborative problem-solving
- Regular feedback and iteration
- Flexibility and adaptability

No single role can succeed in isolation - teamwork is essential!